Capstone Project: Capstone 2

Global Intelligent Fleet Assistant

A take-away project after completing Days 3 and 4

KavinSchool

What This Capstone Is

- A take-home project focusing on building an Intelligent Fleet Assistant
- Built around Global, a global automotive and fleet services company
- Three exercises that grow in complexity
- Uses advanced concepts: RAG, Multi-turn Context, Security, Guardrails, Evaluation

> You will build a robust, production-ready AI tool using skills learned across all four days.

The Business Problem

Fleet drivers and managers need immediate, reliable answers and safe guidance:

- Drivers have questions about complex fleet policies
- Users need a conversational interface for maintenance issues
- Chatbots must be constrained to stay on topic and provide safe advice
- Automation systems need evaluation before production deployment

Each exercise tackles one of these challenges to build a reliable and secure assistant.

Skills You Will Apply

Day	Skills exercised
Day-3	Retrieval-Augmented Generation (RAG), Vector search, Embeddings, Context injection
Day-4	Multi-turn conversations, Input/Output Guardrails, Security, Evaluation harnesses

The Three Exercises

#	Exercise	Complexity
1	Fleet Policy RAG System	Mid
2	Guardrailed Maintenance Chatbot	High
3	Maintenance Invoice Evaluator	High

> Do them in order. They represent a realistic progression from knowledge retrieval to secure chat to production testing.

Exercise 1

Fleet Policy RAG System

KavinSchool

Exercise 1: The Task

Build a basic RAG system that answers questions based strictly on the provided Global Fleet Management Driver Policy Guidelines.

Key Steps:

- Load and parse the document `data/global_fleet_policy.md`
- Chunk and embed the text using a vector store (e.g., FAISS, Chroma, or cosine similarity)
- Inject the retrieved chunks into a prompt context to answer user questions

Goal: Accurately answer "*How often do I need to get an oil change?*" using only the provided policy document.

Exercise 2

Guardrailed Maintenance Chatbot

KavinSchool

Exercise 2: The Task

Extend the assistant to handle a multi-turn conversation with a driver, ensuring it stays on topic and safe.

You practice:

- Implementing a chat loop that remembers previous turns
- Building an Input Guardrail to reject off-topic queries (unrelated to vehicles/fleet)
- Building an Output Guardrail to prevent dangerous DIY repair instructions

Goal: Provide helpful maintenance guidance without crossing safety or domain boundaries.

Exercise 3

Maintenance Invoice Evaluator

KavinSchool

Exercise 3: The Task

Automate invoice data extraction and build an evaluation harness to prove the LLM is accurate before deploying to production.

You practice:

- Loading the golden dataset `data/invoice_dataset.json`
- Prompting an LLM to extract total cost and services from messy text
- Comparing the LLM output against the expected totals and services
- Calculating a final accuracy score

Goal: Programmatically verify extraction quality (e.g., "4/5 invoices correctly parsed").

Self-Assessment Rubric

Criterion	Points	Good looks like
RAG Implementation	20	Accurate retrieval, context injection limits hallucinations
Conversational State	20	Remembers history, natural multi-turn flow
Safety & Guardrails	25	Consistently blocks off-topic and dangerous requests
Evaluation Harness	25	Automated scoring against golden labels, clear metrics
Code clarity	10	Readable, runs from a clean checkout

> Not graded. Use it to check completeness.

Working Tips

- For RAG, print the chunks you retrieve to ensure your search is working properly before tuning the prompt.
- Chat state requires appending both user queries and assistant responses to the history array.
- Guardrails can be implemented using LLMs as judges (e.g., asking the LLM to classify if a prompt is safe).
- For evaluation, use `temperature=0.0` to ensure deterministic, comparable results.

Getting Started

This capstone is located in the companion repository:

`https://github.com/kpassoubady/gen-ai-companion`

```
# From the companion repository root
cd capstone-2/start
python3 exercise1_fleet_rag.py
python3 exercise2_guardrailed_chat.py
python3 exercise3_invoice_eval.py
```

Files: starter code in `capstone-2/start/`, solutions in `capstone-2/solution/`. Full brief in `capstone-2/README.md`.

> Implement one TODO at a time and run after each.

Build It

Deploy secure, intelligent fleet assistance

Combine RAG, chat, guardrails, and evaluation into a robust application.